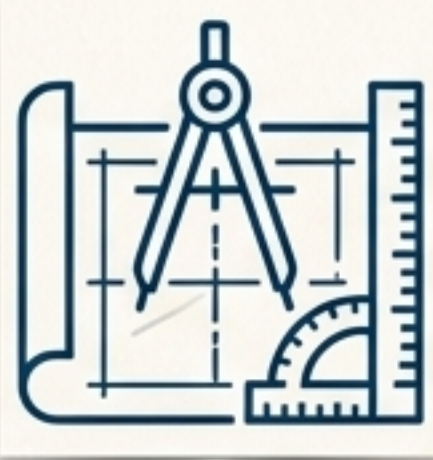


Van Concept naar Code: Een Gids voor Robuuste en Veilige Databases

De essentiële stappen van ontwerp tot beveiliging in MySQL



1. Blauwdruk

Het logisch ontwerp en de normalisatie van data.



2. Fundament

De structuur bouwen met SQL Data Definition Language (DDL).



3. Gebruik

Data manipuleren en bevragen met SQL DML.



4. Vestingwerken

De database beveiligen via toegangsbeheer en hashing.

Een solide database is geen toeval; het is het resultaat van een gedisciplineerd proces. We volgen de levenscyclus van een database, van het eerste idee tot een veilige en performante implementatie. Deze presentatie is uw blauwdruk.

Alles Begint met een Solide Blauwdruk

Bouw

Fysiek Ontwerp

Logisch Ontwerp

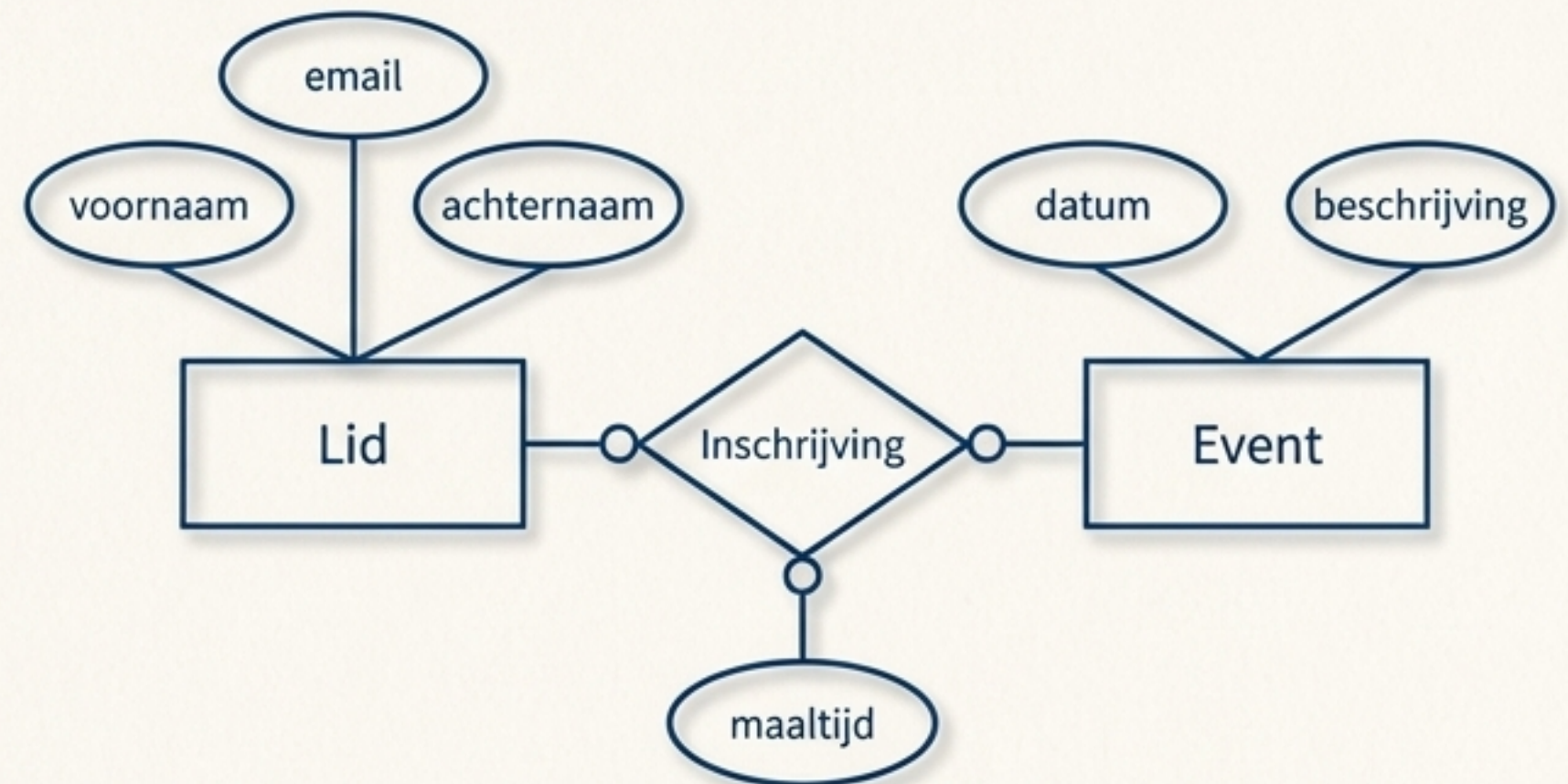
Analyse

Kosten om fouten te herstellen

Een informatiesysteem doorloopt vier fasen. Een fout in een vroege fase is exponentieel duurder om later te herstellen. Het Logisch Ontwerp is de cruciale vertaalslag van bedrijfsprocessen naar een datastructuur.

Het Entity-Relationship Diagram (ERD)

Het ERD is ons belangrijkste instrument. Het visualiseert de bouwstenen van onze data:



Relaties Definiëren en Records Uniek Maken

Kardinaliteit

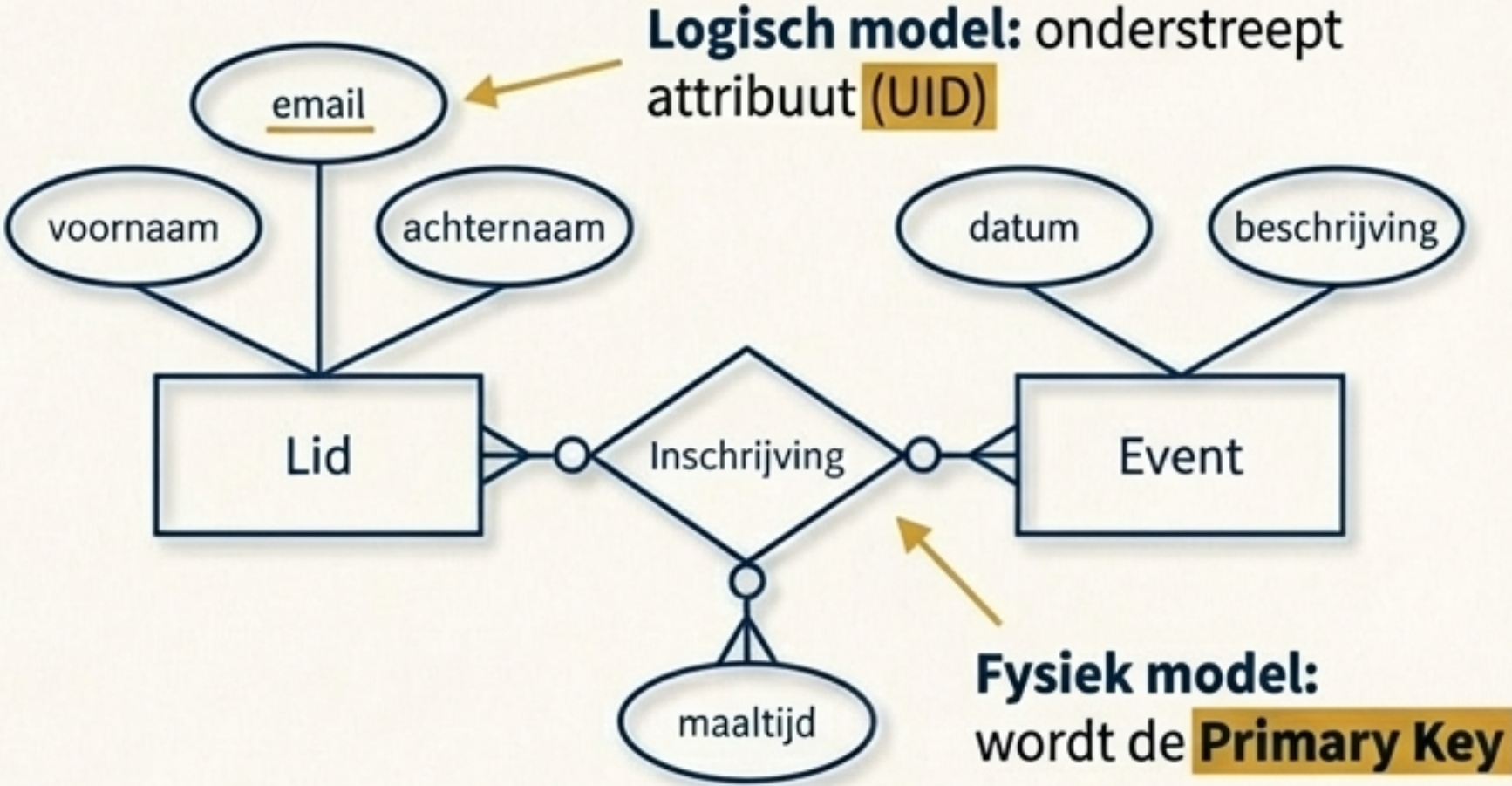
Kardinaliteit bepaalt *hoeveel* entiteiten aan elkaar gerelateerd zijn. We gebruiken de ‘kraaienpootnotatie’:

elke A heeft 0 of 1 B's	
elke A heeft precies 1 B	
elke A heeft 0 of meer B's	
elke A heeft 1 of meer B's	

In ons voorbeeld: ‘Elk Lid heeft zich ingeschreven voor 0 of meer Events’ en ‘Elk Event heeft 0 of meer ingeschreven Leden’.

Unieke Identificatie (UID)

Elke record in een tabel moet uniek zijn. Dit wordt gegarandeerd door een UID.



Waarom Normalisatie Essentieel is: Van Chaos naar Structuur

Probleem: Redundantie

lidnummer	voornaam	familienaam	ploeg	coach
1	Lois	Jansen	Team A	J. Jansen
2	Mary	Growens	Team A	J. Jansen
3	Kare	Jansen	Team A	J. Jansen
4	Ean	Jansen	Team A	J. Jansen
5	Edm	Kartill	Team A	J. Jansen
6	Martin	Donan	Team A	J. Jansen
7	Lois	Jansen	Team A	J. Jansen
8	Mary	Growens	Team A	J. Jansen
9	Kare	Jansen	Team A	J. Jansen
10	Eam	Kartiï	Team A	J. Jansen

- Wanneer gegevens onnodig herhaald worden, leidt dit tot:
 - Meer opslagruimte.
 - Inconsistenties (bv. een naam op twee plaatsen anders gespeld).
 - Moeilijk onderhoud.

Oplossing: Normalisatie (3NF)

leden					
#	lidnummer	voornaam	familienaam	ploeg_id	
	1	Lois	Jansen	1	
	2	Mary	Growens	1	
	3	Kare	Jansen	2	
	...	...	...	...	

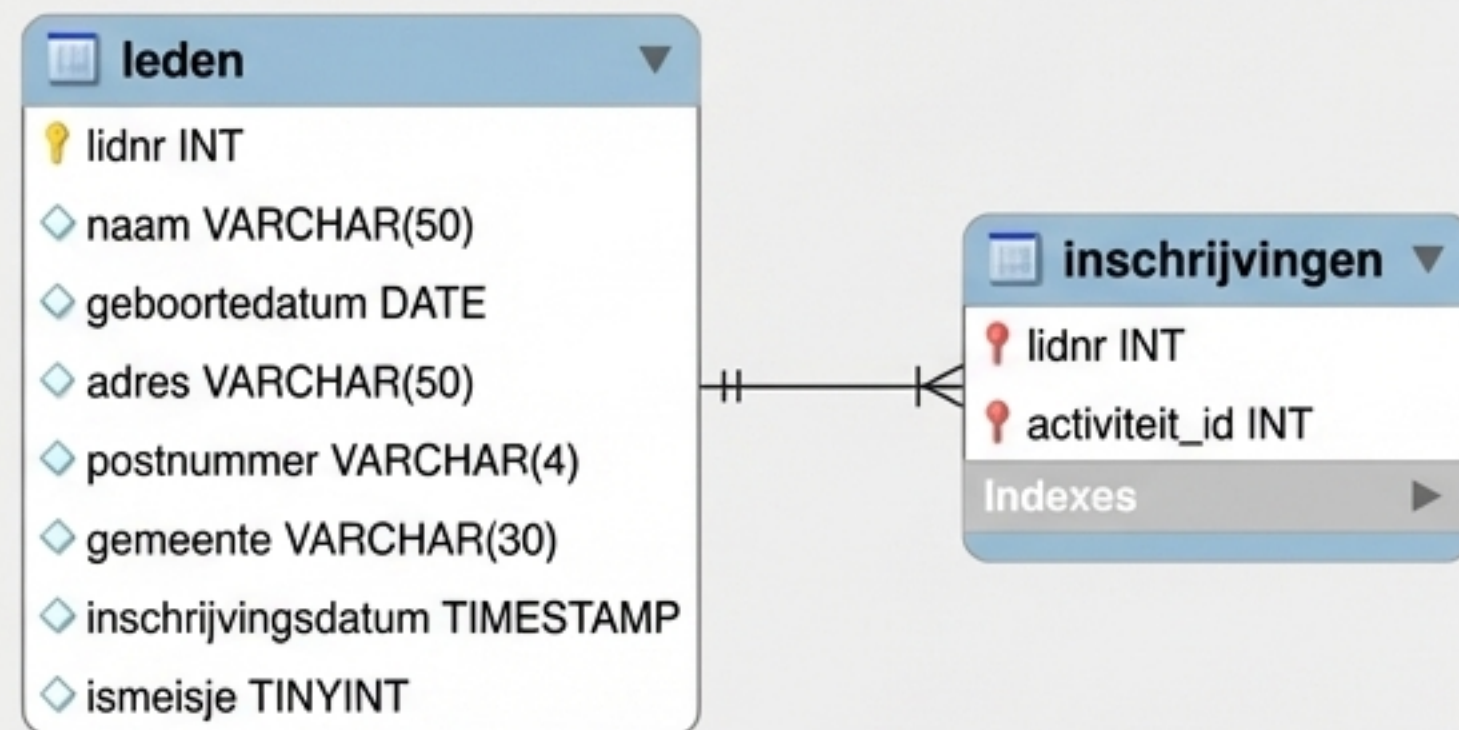
ploegen		
#	ploeg_id	coach
	1	J. Jansen
	2	J. Jansen

****Foreign Key****

- Normalisatie is het proces van het opsplitsen van tabellen om redundantie te elimineren.
- **1NF**: Geen herhalende groepen.
- **2NF**: Volledig afhankelijk van de primaire sleutel.
- **3NF**: Niet afhankelijk van andere niet-sleutel attributen.

Het Fundament Leggen met SQL Data Definition Language (DDL)

Met DDL vertalen we ons logische model naar een fysieke databasestructuur.



Het 'CREATE TABLE' statement

```
CREATE TABLE leden (
  lidnr INT NOT NULL AUTO_INCREMENT,
  naam VARCHAR(50) NOT NULL,
  geboortedatum DATE,
  ismeisje TINYINT DEFAULT 0,
  PRIMARY KEY (lidnr)
);
```

Datatype: cruciaal voor performance en integriteit.

Constraint: dwingt af dat een veld een waarde moet bevatten.

Definieert de unieke identificatie.

```
CREATE TABLE inschrijvingen (
  lidnr INT NOT NULL,
  activiteit_id INT NOT NULL,
  PRIMARY KEY (lidnr, activiteit_id),
  FOREIGN KEY (lidnr) REFERENCES leden(lidnr)
);
```

Garandeert referentiële integriteit.

De Data Tot Leven Brengen: Werken met SQL DML

DML is de gereedschapskist om data te creëren, lezen, updaten en verwijderen (CRUD).

Andere DML Commando's



INSERT

Voegt nieuwe records toe.

```
INSERT INTO leden (naam,  
ismeisje) VALUES ("Marie", 1);
```



UPDATE

Wijzigt bestaande records.



DELETE

Verwijdert records.

```
-- Vraagt data op uit de 'leden' tabel  
SELECT lidnr, voornaam, achternaam, geslacht  
FROM leden  
WHERE gemeente = "Waregem";
```

Projectie
(kolommen kiezen)

Selectie
(rijen filteren)



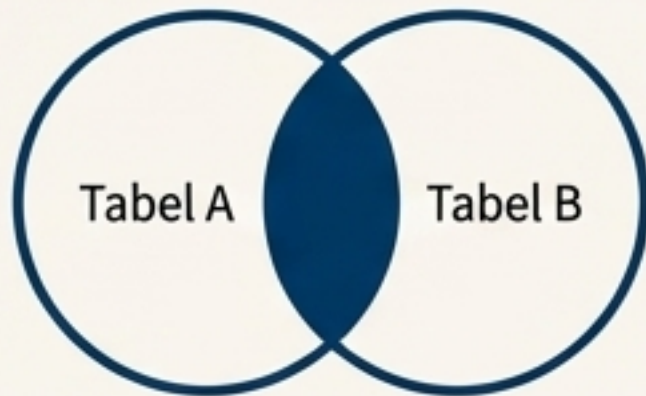
lidnr	voornaam	achternaam	geslacht
2913	Hanna	de Vries	v
3362	Marie	van Doorn	v

Meer dan de Som der Delen: Data Verbinden met Joins

Een `JOIN` combineert rijen uit twee of meer tabellen op basis van een gerelateerde kolom (de Foreign Key).

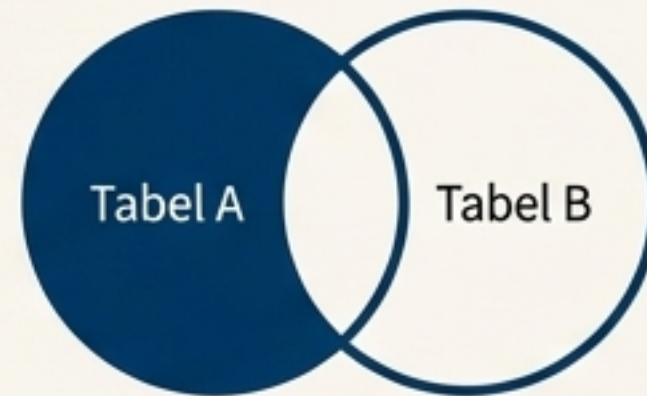
De Belangrijkste Soorten Joins

INNER JOIN



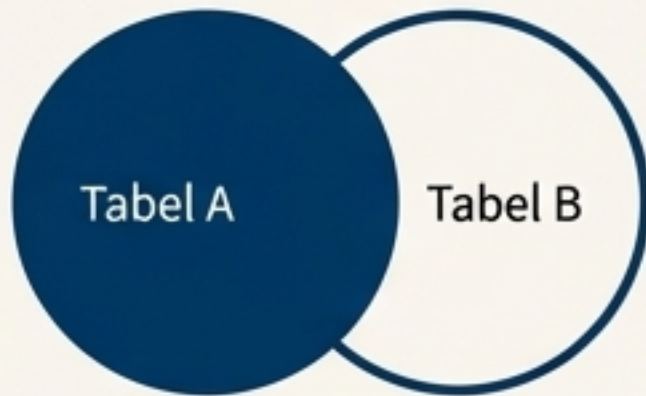
Geeft alleen de records terug die een match hebben in *beide* tabellen.

LEFT JOIN



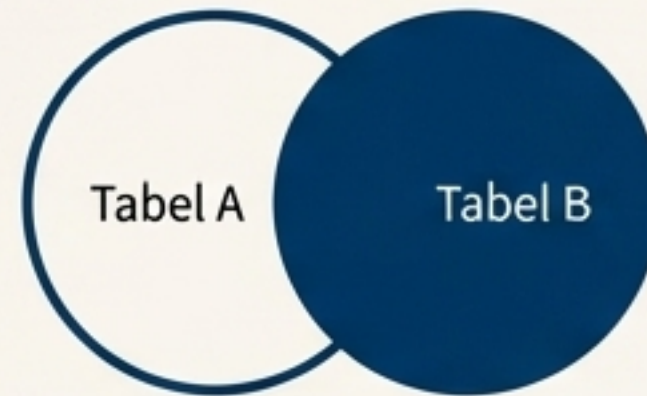
Geeft *alle* records uit de linkertabel terug, en de gematchte records uit de rechtertabel.

LEFT JOIN

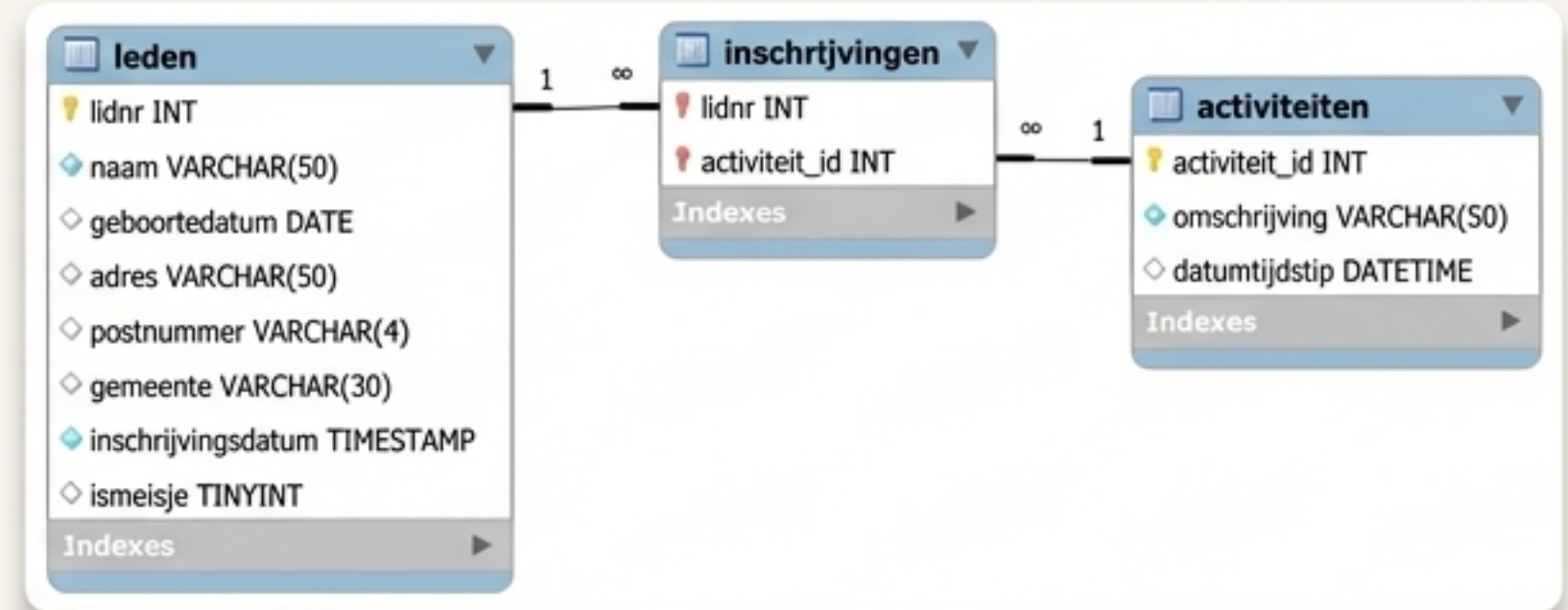


Geeft *alle* records uit de linkertabel terug, en de gematchte records uit de rechtertabel.

RIGHT JOIN



Geeft *alle* records uit de rechtertabel terug, en de gematchte records uit de linkertabel.

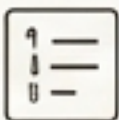


SQL Voorbeeld ('INNER JOIN')

```
SELECT l.naam, a.omschrijving
FROM leden AS l
INNER JOIN inschrijvingen AS i
    ON l.lidnr = i.lidnr
INNER JOIN activiteiten AS a
    ON i.activiteit_id = a.activiteit_id;
```


Van Rijen naar Inzichten: Data Aggregeren en Groeperen

Statistische (of aggregatie-) functies vatten data samen over meerdere rijen tot één resultaat.



Veelgebruikte Functies:

COUNT(), SUM(), AVG(), MIN(), MAX()

Before

gemeente	ismeisje
Nazareth	1
Tielt	0
Tielt	0
Tielt	0
Tielt	1
Tielt	1
Tielt	1
Tielt	1
Veurne	0

After

gemeente	aantal_leden
Tielt	4
Tielt	4
Veurne	3
Veurne	3
Waregem	5
Waregem	4

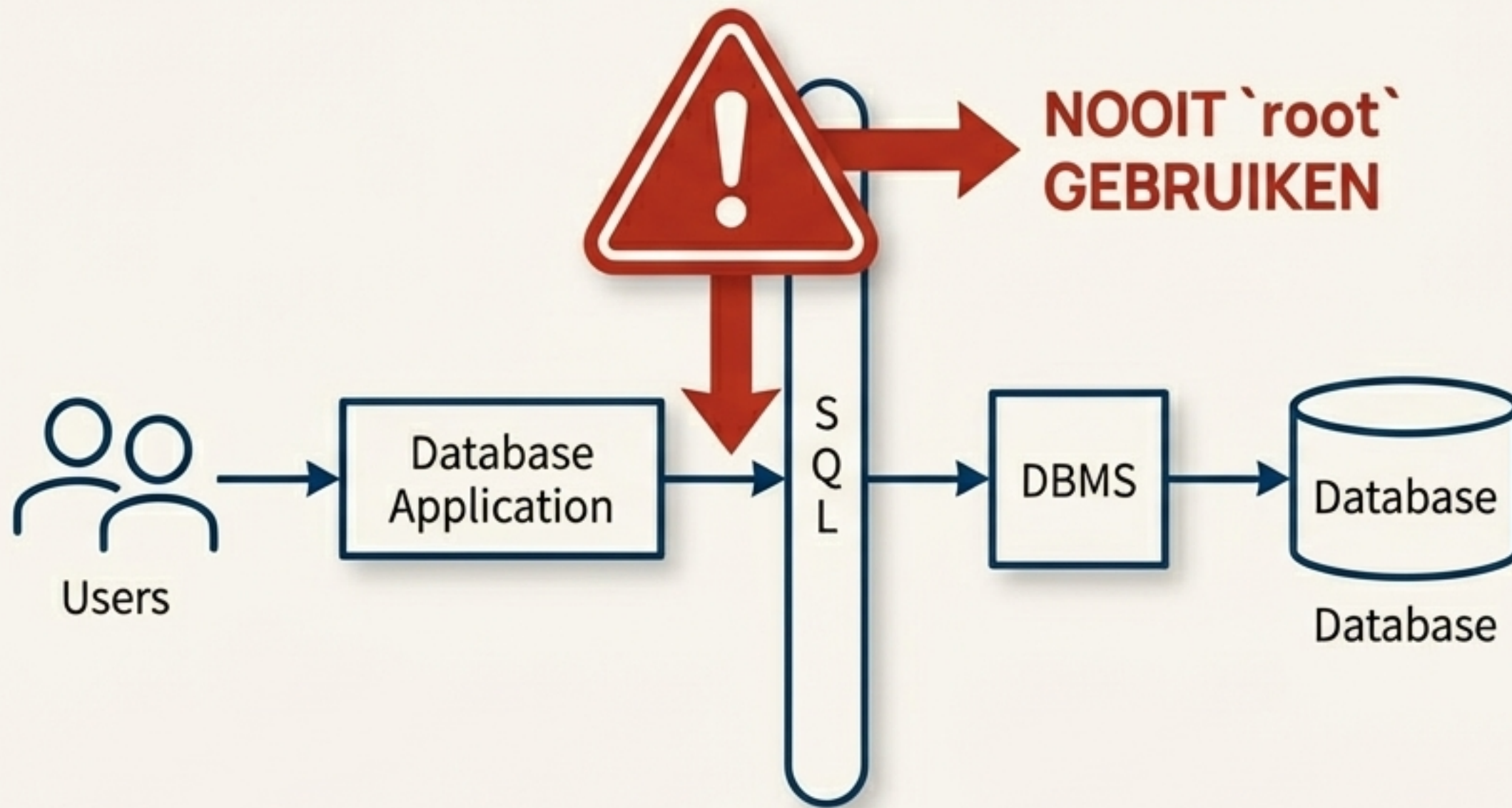
```
SELECT gemeente, COUNT(lidnr) AS aantal_leden
FROM leden
GROUP BY gemeente
HAVING COUNT(lidnr) > 1;
```

GROUP BY: Combineert rijen met dezelfde waarden tot samenvattende rijen.

HAVING: Filtert op het resultaat van de aggregatiefunctie.

De Vesting Bouwen: De Noodzaak van Toegangsbeheer

Een applicatie die verbinding maakt met de database mag **nooit** de almachtige `root`-gebruiker gebruiken.



Het Probleem met `root`

- De `root`-gebruiker heeft volledige rechten: `CREATE`, `DROP`, `DELETE` op *elke* database.
- Als de connectiegegevens lekken, heeft een aanvaller de sleutels tot het hele koninkrijk. Dit is een catastrofaal risico.

De Oplossing: Principe van Minimale Rechten (Least Privilege)

- Creëer een aparte, specifieke MySQL-gebruiker voor elke applicatie.
- Geef deze gebruiker *enkel* de rechten die absoluut noodzakelijk zijn.

Minimale Rechten in de Praktijk: Een Veilige Gebruiker Creëren

Met DCL-queries (Data Control Language) beheren we de toegangsrechten tot onze database.

De Drie Stappen

1. Gebruiker Aanmaken:

Creëer een nieuwe gebruiker die alleen vanaf een specifieke host mag verbinden.

```
CREATE USER "appuser"@"localhost"  
IDENTIFIED BY "sterkWachtwoord123!";
```

2. Rechten Toekennen:

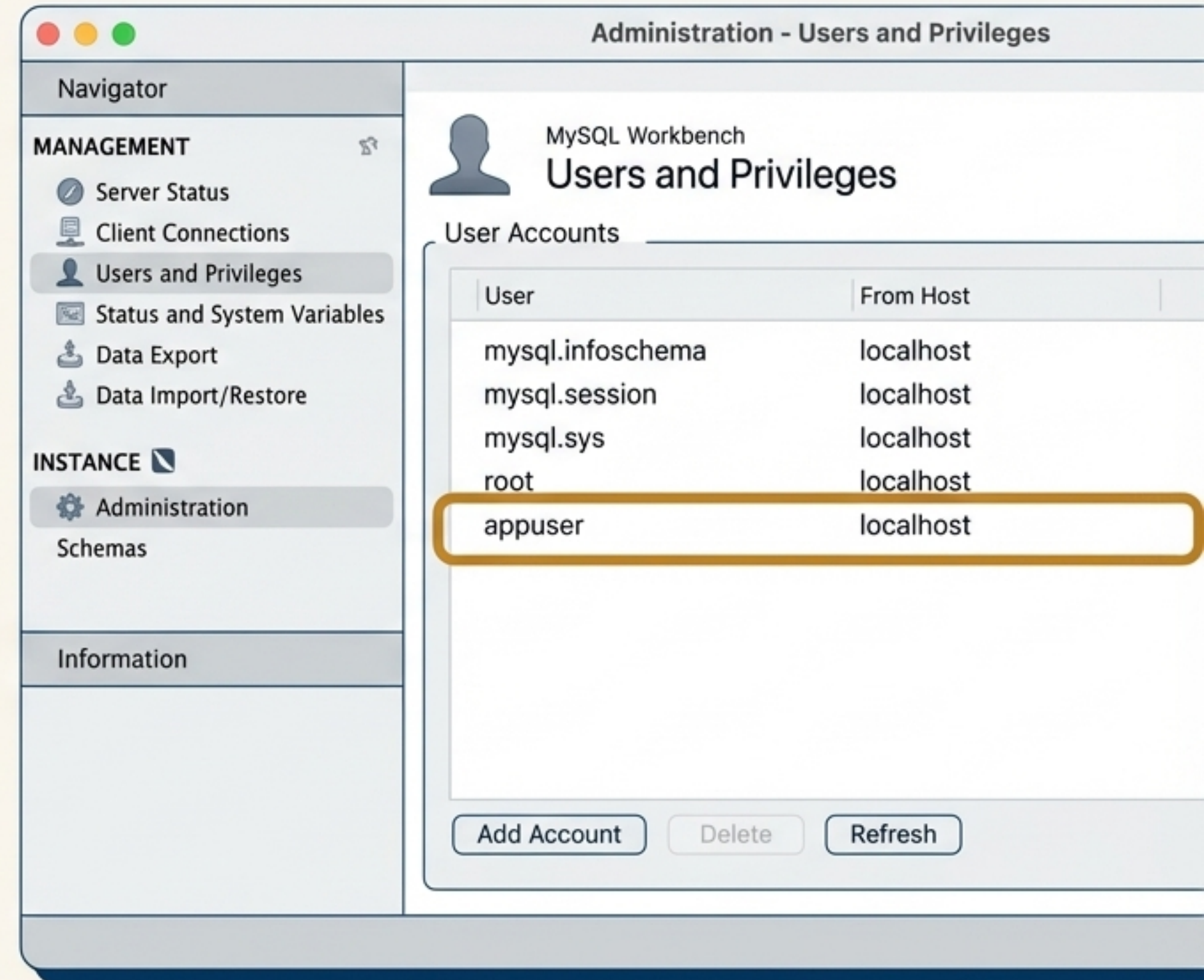
Ken specifieke DML-rechten toe op een specifieke database.

```
GRANT SELECT, INSERT, UPDATE, DELETE  
ON jeugdbeweging.*  
TO "appuser"@"localhost";
```

3. Rechten Toepassen:

Zorg ervoor dat de nieuwe rechten onmiddellijk van kracht worden.

```
FLUSH PRIVILEGES;
```



De Onvergeeflijke Zonde: Wachtwoorden in Plaintext

Wachtwoorden opslaan als leesbare tekst is een fundamentele en onacceptabele beveiligingsfout.



logins			
login_id	gebruikersnaam	wachtwoord	e-mail
1	jdesmet	testje	jan.desmet@gmail.com
2	kvandeveld	mijnwachtwoord	karel.vandeveld@outlook.com

Waarom is dit een ramp?

- **Bij een datalek:** Alle gebruikersaccounts zijn direct gecompromitteerd.
- **Wachtwoordhergebruik:** Een lek op uw site kan leiden tot inbraken op hun e-mail, bank-app, etc.
- **Vertrouwensbreuk:** Als beheerder mag u *nooit* de wachtwoorden van uw gebruikers kunnen inzien.

De Eerste Verdedigingslinie: De Essentie van Hashing

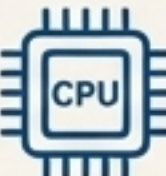
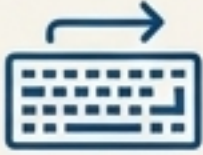
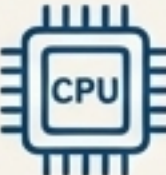
Wat is Hashing?

Een proces dat data (de *plaintext*) via een wiskundig algoritme omzet in een unieke, vaste-lengte string (de *hash* of *digest*).

Cruciale eigenschap: Een hash-functie werkt maar in één richting. Je kunt een hash niet terugrekenen naar de originele plaintext. Dit is het fundamentele verschil met encryptie.



Hoe Werkt Authenticatie?

1.  Gebruiker registreert zich met `wachtwoord123`.
2.  De applicatie berekent de hash (bv. `s5df...zg`) en slaat **enkel de hash** op in de database.
3.  Gebruiker logt in met `wachtwoord123`.
4.  De applicatie berekent opnieuw de hash van de invoer.
5.  = De applicatie vergelijkt de *nieuwe hash* met de *opgeslagen hash*. Als ze identiek zijn, is het wachtwoord correct.

De Verdediging Versterken met 'Salt'

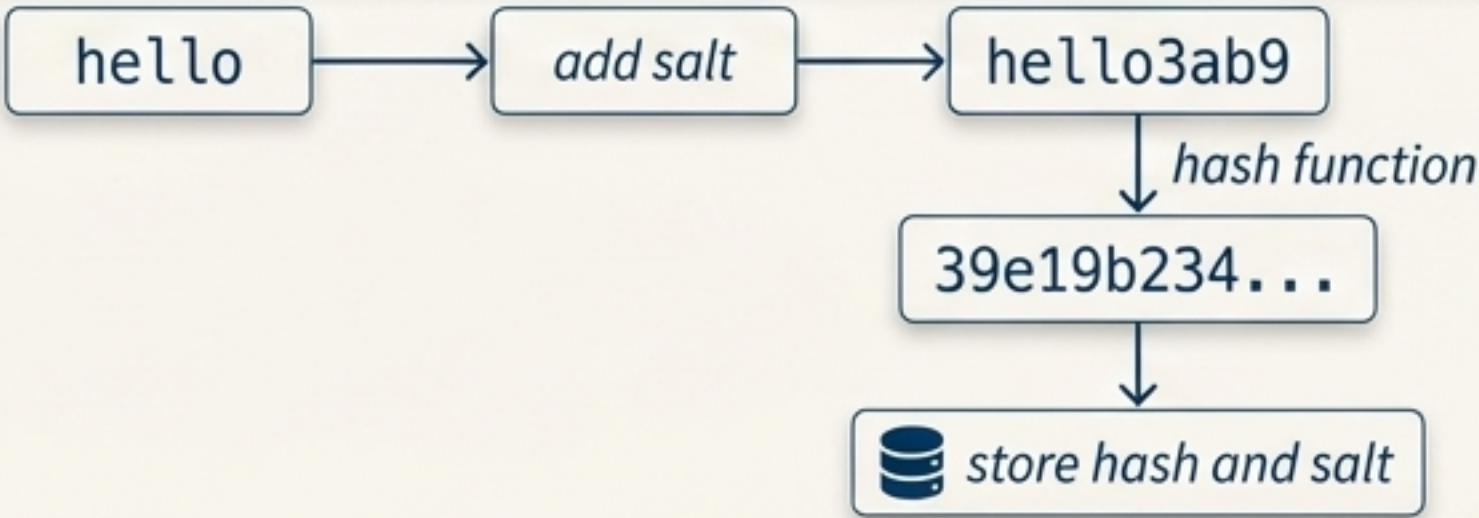
Het Probleem: Rainbow Tables

Zwakke hashing-algoritmes zijn kwetsbaar voor 'rainbow tables': vooraf berekende lijsten van hashes voor veelvoorkomende wachtwoorden.

De Oplossing: Salting

Een **salt** is een unieke, willekeurige tekenreeks die wordt toegevoegd aan het wachtwoord *voordat* het gehasht wordt. Elke gebruiker krijgt een eigen, unieke salt.

	Without Salt		With Salt	
				
Password	p4s5w3rdz	p4s5w3rdz	p4s5w3rdz	p4s5w3rdz
Salt	–	–	et52ed	ye5sf8
Hash	f4c31aa	f4c31aa	1vn49sa	z32i6t0



Van Theorie naar Praktijk: Veilige Logins in C#

Het implementeren van veilige wachtwoordopslag is een gestructureerd proces.

Stap 1: Registratie - Een Salted Hash Creëren

1. Genereer een willekeurige, cryptografisch sterke salt.
2. Voeg de salt samen met het plaintext wachtwoord.
3. Hash de gecombineerde string met een sterk algoritme (bv. SHA-256).
4. Sla de `gebruikersnaam`, `hash` en `salt` op in de database.

```
// Genereer de hash van wachtwoord + salt
public string GetHashedPassword(string input)
{
    //create cryptography class instance
    SHA256 sha256 = SHA256.Create();

    //compute bytes of input string
    byte[] bytes = sha256.ComputeHash(Encoding.Unicode.GetBytes(input));

    //compute string from bytes
    StringBuilder builder = new StringBuilder();
    for (int i = 0; i < bytes.Length; i++)
    {
        builder.Append(bytes[i].ToString("x2"));
    }

    //return hashed string
    return builder.ToString();
}
```

Stap 2: Login - De Invoer Verifiëren

1. Haal de opgeslagen `hash` en `salt` op voor de gebruiker.
2. Voeg de *opgehaalde salt* samen met het *ingevoerde wachtwoord*.
3. Hash deze nieuwe string.
4. Vergelijk de *zojuist berekende hash* met de *opgeslagen hash*.

```
string username = <from login page>
string password = <from login page>

//find user record based on username
//-- your code here
//if user not found, it's invalid login attempt

//retrieve user's Hashed Password and Salt
string hashedPassword = <from user record>
string salt = <from user record>

//compute hash based on password entered by user and salt stored with us
string hashedInput = GetHashedPassword(password + salt)

//compare hashedInput with hashedPassword
if (hashedInput != hashedPassword)
{
    //invalid login ... your code to return error here
}
else
{
    //valid login .. your code may continue to execute
}
```


De Cirkel is Rond: Ontwerp, Bouw en Veiligheid zijn Verbonden

Samenvatting van de Reis

- We zijn gestart met een **Blauwdruk**.
- We hebben het **Fundament** gelegd.
- We hebben de **Kracht** van data ontsloten.
- We hebben de **Vesting** gebouwd.



De Centrale Les

Elke stap bouwt voort op de vorige. Een goed, genormaliseerd ontwerp is niet alleen efficiënter, het is ook inherent makkelijker te beveiligen.

Een robuuste database is een ecosysteem waarin ontwerp, implementatie en beveiliging elkaar versterken.